เฉลยใบงานที่ 1 และ 2 สำหรับผู้สอน

ผลที่ควรได้ ตารางบันทึกผล และแนวคำตอบ (ไม่แจกนักศึกษา)

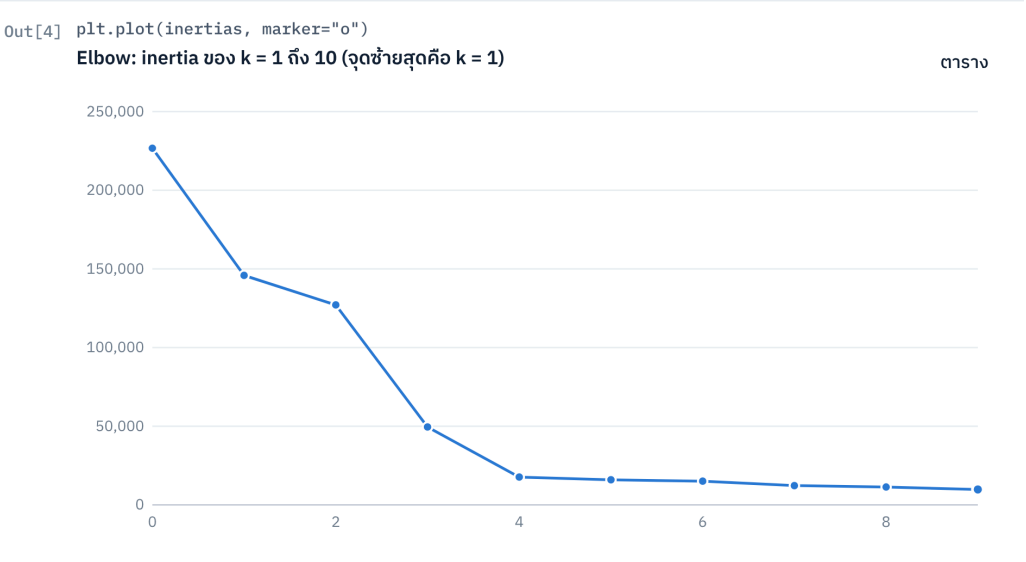
ตัวเลขทุกค่าได้จากการรันเฉลยใน ThaiPyBlocks และตรงกับโค้ด Python ที่แอปสร้างเมื่อรันด้วย pandas 3.0 และ scikit-learn 1.8 (รุ่นอื่นอาจต่างที่ทศนิยมตำแหน่งท้าย ๆ แต่ข้อสรุปเหมือนเดิม) เปิดเฉลยบล็อกได้ที่เมนู ⋯ → เปิดโครงงาน → `solution_kmeans.json` / `solution_neural_network.json` แล้วอัปโหลด CSV และกด ▶ รัน

1 ใบงานที่ 1 K-Means (test_size 0.2, random_state 42)

ตารางที่ 1 inertia ของ k = 1–10

k	inertia	ลดลงจาก k ก่อนหน้า
1	226,726.5	–
2	145,867.4	80,859.1
3	127,108.7	18,758.7
4	49,493.0	77,615.7
5	17,688.4	31,804.6
6	15,921.9	1,766.5
7	15,089.3	832.6
8	12,249.1	2,840.2
9	11,332.2	916.9
10	9,802.4	1,529.8

ข้อศอกอยู่ที่ **k = 5**: จาก k = 4 ลดลง 31,804.6 แต่ถัดไปลดลงแค่ 1,766.5



กราฟ elbow ที่ควรได้ (จุดที่ 4 นับจาก 0 คือ k = 5)

ตารางที่ 2 จำนวนกลุ่มกับคุณภาพการจัดกลุ่ม

k	Silhouette ชุดฝึก	Silhouette ชุดทดสอบ
3	0.399	0.509
4	0.593	0.677
5	**0.704**	0.651
6	0.626	0.557

k = 5 ดีที่สุดบนชุดฝึก แต่ชุดทดสอบให้ k = 4 สูงกว่า เพราะชุดทดสอบมีลูกค้ากลุ่มรายได้น้อย ใช้จ่ายสูงเพียง 2 คน (k = 4 รวมกลุ่มนี้เข้ากับกลุ่มกลาง) ค่าจากลูกค้า 40 คนจึงแกว่งง่าย ใช้ประกอบคำถามข้อ 3

ตารางที่ 3 ตั้งชื่อกลุ่มลูกค้า (k = 5)

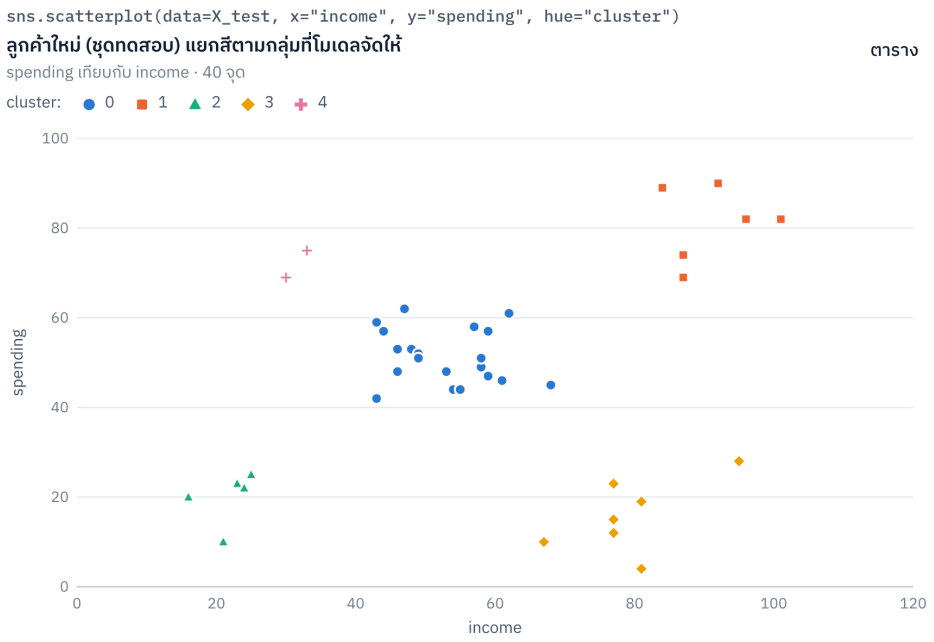
cluster	income เฉลี่ย (พันบาท)	spending เฉลี่ย	ชื่อกลุ่ม (ตัวอย่าง)
0	55.2	50.2	รายได้กลาง ใช้จ่ายกลาง
1	87.9	82.9	รายได้สูง ใช้จ่ายสูง
2	23.1	19.3	รายได้น้อย ใช้จ่ายน้อย
3	88.3	16.2	รายได้สูง ใช้จ่ายน้อย
4	24.2	84.2	รายได้น้อย ใช้จ่ายสูง

ตารางที่ 4 เปลี่ยนวิธีแบ่งข้อมูล (k = 5)

test_size	random_state	แถว ฝึก / ทดสอบ	Silhouette ชุดฝึก	Silhouette ชุดทดสอบ	ลูกค้าใหม่ (30, 85)
0.2	42	160 / 40	0.704	0.651	กลุ่ม 4
0.2	7	160 / 40	0.706	0.658	กลุ่ม 2
0.5	42	100 / 100	0.712	0.682	กลุ่ม 4

ทั้งสามแถวคือกลุ่ม "รายได้น้อย ใช้จ่ายสูง" เลขกลุ่มเป็นแค่ป้ายที่ K-Means ตั้งใหม่ทุกครั้งที่ฝึก

Out[8]
```
sns.scatterplot(data=X_test, x="income", y="spending", hue="cluster")
```



ลูกค้าในชุดทดสอบ 40 คน แยกสีตามกลุ่มที่โมเดล k = 5 จัดให้

แนวคำตอบใบงานที่ 1

1. เห็น 5 กลุ่ม (สี่มุมกับตรงกลาง) ตรงกับข้อศอก k = 5
2. ชุดทดสอบคือลูกค้าที่โมเดลไม่เคยเห็น Silhouette ชุดทดสอบ 0.651 ใกล้ชุดฝึก 0.704 จึงบอกได้ว่ากลุ่มใช้กับลูกค้าใหม่ได้ ถ้าใช้ทั้ง 200 คนฝึกจะไม่มีข้อมูลเหลือไว้ตรวจ
3. ชุดทดสอบให้ k = 4 แต่ต้องเลือก k จากชุดฝึก (k = 5) การเลือกด้วยชุดทดสอบคือเอาข้อสอบมาใช้ตอนฝึก คะแนนทดสอบจะดีเกินจริง และลูกค้า 40 คนน้อยเกินกว่าจะใช้ตัดสินเอง
4. ไม่ได้ย้าย เลขกลุ่มเปลี่ยนเพราะฝึกใหม่ด้วยชุดฝึกคนละชุด ค่าเฉลี่ยของกลุ่มบอกว่าเขาอยู่กลุ่ม "รายได้น้อย ใช้จ่ายสูง" ทุกครั้ง เหมาะกับโปรโมชันผ่อน 0% หรือคูปองส่วนลด

2 ใบงานที่ 2 Neural Network (test_size 0.25 แบบ stratify, random_state 42)

ตารางที่ 1 จำนวนถาดงอก / ไม่งอก

	งอก	ไม่งอก	รวม	สัดส่วนงอก
ทั้งไฟล์	164	236	400	0.41
ชุดทดสอบ	41	59	100	0.41

stratify ทำให้สัดส่วนงอกเท่ากันทั้งสองชุด การทาย "ไม่งอก" ทุกถาดได้ accuracy 0.59 บนชุดทดสอบ

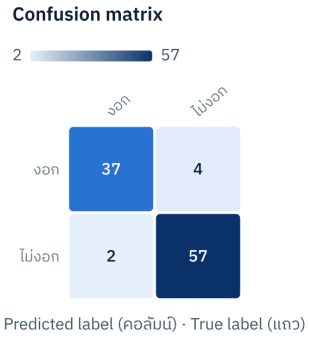
ตารางที่ 2 เปรียบเทียบโมเดล

โมเดล	hidden_layer_sizes	max_iter	ชุดฝึก	ชุดทดสอบ	ฝึก − ทดสอบ	คำเตือน
LogisticRegression	–	1000	0.450	0.460	−0.010	ไม่มี
MLP	(2,)	1000	0.590	0.590	0.000	ไม่มี*
MLP	(8,)	1000	0.947	0.910	0.037	ConvergenceWarning
MLP	**(16,)**	1000	0.963	**0.940**	0.023	ไม่มี
MLP	(100,)	1000	0.970	0.920	0.050	ไม่มี
MLP	(16, 16)	1000	0.977	0.920	0.057	ไม่มี
MLP	(16,)	300	0.913	0.880	0.033	ConvergenceWarning
MLP ไม่ปรับสเกล	(16,)	1000	0.747	0.780	−0.033	ไม่มี

* ใน Colab จะมี UndefinedMetricWarning จาก classification_report เพราะไม่มีถาดใดถูกทายว่างอก

ตารางความสับสนของ MLP (16,): งอก → งอก 37, งอก → ไม่งอก 4, ไม่งอก → งอก 2, ไม่งอก → ไม่งอก 57 · แถว งอก ใน classification_report: precision 0.95 recall 0.90 f1 0.93

```
Out[8] ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap="Blues")
```



ตารางความสับสนที่ควรได้ (MLP 16 นิวรอน)

แนวคำตอบใบงานที่ 2

1. ได้ 0.59 LogisticRegression ได้ 0.46 แย่กว่าการทายแบบนั้น เพราะถาดงอกรวมเป็นวงรีกลางกราฟและถูกล้อมด้วยถาดไม่งอก เส้นตรงเส้นเดียวแบ่งไม่ได้
2. โมเดลทาย "ไม่งอก" ทุกถาด แถว งอก มี precision และ recall เป็น 0.00 นิวรอน 2 ตัวน้อยเกินจะสร้างเส้นแบ่งโค้ง (underfitting)
3. (16,) ได้ชุดทดสอบสูงสุด 0.94 ส่วน (16, 16) ได้ชุดฝึกสูงสุด 0.977 แต่ชุดทดสอบเหลือ 0.92 ส่วนต่างที่กว้างขึ้น (0.023 → 0.057) คือสัญญาณว่าโมเดลเริ่มจำข้อมูลฝึก (overfitting)
4. ฝึกครบ 300 รอบแล้วค่า loss ยังลดไม่สุด (ยังไม่ลู่เข้า) accuracy จึงต่ำกว่า (0.913 / 0.880) แก้โดยเพิ่ม max_iter เป็น 1000
5. ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานต้องมาจากชุดฝึกเท่านั้น ไม่เช่นนั้นข้อมูลทดสอบรั่วเข้าการฝึก เมื่อไม่ปรับสเกล accuracy ตกเหลือ 0.747 / 0.780 เพราะ Neural Network ไวต่อขนาดตัวเลขของแต่ละคอลัมน์
6. แปลงค่าใหม่ด้วย scaler.transform ตัวเดิมก่อนทำนาย หรือใช้บล็อก "ไปป์ไลน์" รวม StandardScaler กับ MLPClassifier ให้ปรับสเกลอัตโนมัติ